Übungsblatt 8: I/O und Systemarchitektur

Mikrocomputertechnik 1

Aufgabe 1: Die Gerätekarte der Pumpenstation (10 Punkte)

Eine kleine Pumpenstation füllt einen Wassertank; sie begleitet Sie durch das ganze Blatt. Ihr RV32I-Mikrocontroller liest zwei Schwimmerschalter und einen Pumpentaster und zeigt den Füllstand auf einer LED-Zeile mit vier Pixeln an. Beide Geräte erreicht er über Memory-Mapped I/O. Die Karte ist eine Annahme dieses Blatts, keine Ripes-Konfiguration.

Register	Adresse	Zugriff	Vertrag
Schalterwort	`0xE0000000`	lesen	Bit 0: Schwimmer unten, Bit 1: Schwimmer oben, Bit 2: Pumpentaster; aktiv High, 1 = nass bzw. gedrückt
Pixel 0 bis 3	ab `0xE0000010`	schreiben	je ein Farbwort `0x00RRGGBB`, Abstand vier Byte

Weitere Bits des Schalterworts gehören zu anderen Schaltern und können ebenfalls gesetzt sein.

1. Ergänzen Sie die Pixeladressen. Die erste Spalte ist ein Beispiel. Darf ein Programm an die letzte Adresse einen „Pixel 4“ schreiben? (2 P)

Pixel	0	1	3	erste Adresse dahinter
Byteoffset	0			
Adresse	`0xE0000010`			

2. Ergänzen Sie die Farbtabelle. Gelb entsteht aus Rot und Grün. Die erste Spalte ist ein Beispiel. (2 P)

Farbe	Rot	Gelb	Grün	Blau	Schwarz
RR GG BB	FF 00 00	FF FF 00		00 00 FF	
Farbwort	`0x00FF0000`		`0x0000FF00`		`0x00000000`

3. Pixel 1 erhält das grüne Farbwort. Welche Bytes liegen in der Little-Endian-Umgebung an den vier Adressen ab Pixel 1? An welcher Adresse liegt das Byte GG? (1 P)

4. Der Adressdecoder erkennt den Gerätebereich an der obersten Hexziffer E = 1110_2. Geben Sie das Auswahlsignal S_E als Verknüpfung von A_{31} bis A_{28} an. Tipp: In der Vorlesung gilt für F: $S_F = A_{31} \wedge A_{30} \wedge A_{29} \wedge A_{28}$. (1 P)
5. Ergänzen Sie die Tabelle; $A_{31..28}$ steht für $A_{31}A_{30}A_{29}A_{28}$. Die erste Spalte ist ein Beispiel. (2 P)

Adresse	`0xE0000000`	`0xE000001C`	`0xF0000010`	`0x60000010`	`0x0E000010`
$A_{31..28}$	1110				
S_E	1				

6. Wie viele Byteadressen wählt $S_E = 1$ aus? Geben Sie die Zahl als Zweierpotenz und in MiB an (1 MiB $= 2^{20}$ Byte). Welches einzelne Adressbit ist beim Schalterwort 0 und bei allen vier Pixeln 1? Tipp: `0x10` = $1\,0000_2$. (2 P)

Aufgabe 2: Den Füllstand anzeigen (12 Punkte)

Pixel 1 zeigt den Füllstand. Die beiden Schwimmerbits 1:0 bestimmen die Farbe:

Bits 1:0	Schwimmer	Zustand	Farbe
00	beide trocken	leer	Rot
01	nur unten nass	halb	Gelb
11	beide nass	voll	Grün
10	nur oben nass	Sensorfehler	Blau

Das Programm verwendet die Karte aus Aufgabe 1:

```
        li   s1, 0xE0000000      # Schalterwort
        li   s2, 0xE0000010      # Pixelbasis
loop:   lw   t0, 0(s1)           # Rohwert lesen
        andi t1, t0, 3           # Schwimmerbits isolieren
        beq  t1, zero, leer
        li   t3, 1
        beq  t1, t3, halb
        li   t3, 3
        beq  t1, t3, voll
        li   t2, 0x000000FF      # sonst Sensorfehler: Blau
        j    zeigen
leer:   li   t2, 0x00FF0000      # Rot
        j    zeigen
halb:   li   t2, 0x00FFFF00      # Gelb
        j    zeigen
voll:   li   t2, 0x0000FF00      # Grün
zeigen: sw   t2, 4(s2)           # Pixel 1
        j    loop
```

1. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. Tipp: Nach dem `andi` zählen nur noch die Bits 1:0. (4 P)

Rohwert (Bits 7:0)	t1 nach `andi`	Zustand	Farbwort an Pixel 1
`0000 0001`	1	halb	`0x00FFFF00`
`0000 0011`			
`0000 0110`			
`0000 0100`			
`1000 0101`			

2. Verfolgen Sie das Programm für den Rohwert `0000 0110`. Welche der drei `beq` springen? Welches Wort schreibt `sw` an welche Adresse? (2 P)
3. Ein Entwurf schreibt versehentlich `sw t1, 4(s2)`. Welches Wort erhält Pixel 1 bei vollem Tank, und was sieht man? (1 P)

Fehlersuche. Ein Team hat das Programm abgetippt. Bei vollem Tank zeigt Pixel 1 Grün. Sobald jemand

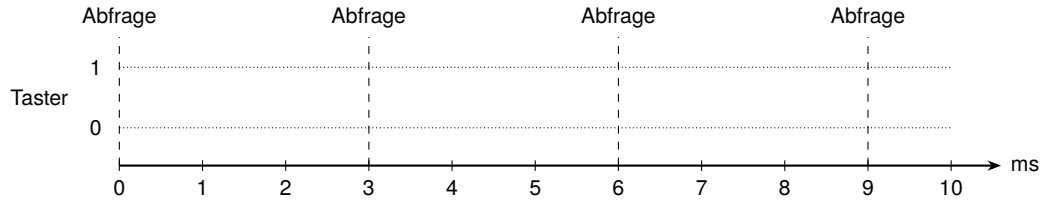
den Pumpentaster drückt, wird Pixel 1 blau, obwohl beide Schwimmer nass bleiben.

4. Welche Bits 2:0 liest die Schleife vor und während des Drucks? Welche Konstante im Programm ist verdächtig? Bestätigen Sie den Verdacht mit dem Wert von t1 vor und während des Drucks. (3 P)
5. Erweiterung: Pixel 0 soll den Pumpentaster anzeigen, gedrückt Weiß (`0x00FFFFFF`), sonst Schwarz. Schreiben Sie die Befehle, die Sie direkt hinter `lw t0, 0(s1)` einfügen; verwenden Sie t4 und t5. Warum muss Ihr Code auch Schwarz ausdrücklich schreiben? (2 P)

Aufgabe 3: Den Pumpentaster abfragen (10 Punkte)

Der Pumpentaster liegt auf Bit 2 des Schalterworts. Das Schalterwort ist ein reines Pegelregister: Es zeigt den gegenwärtigen Zustand, speichert keine Ereignisse und hat keine Lese-Nebenwirkung. Jeder Druck soll die Pumpe genau einmal starten.

1. Die Schleife fragt den Taster alle 3 ms ab. Eine Abfrage kostet 240 Takte bei angenommenen 16 MHz. Wie lange dauert eine Abfrage? Welchen Anteil der Prozessorzeit belegt sie? Wie lange wartet ein gehaltener Druck höchstens auf die nächste Prüfung? (3 P)
2. Die Abfragen liegen bei 0, 3, 6 und 9 ms. Ein Bediener tippt den Taster nur von 3,5 ms bis 5,0 ms. Zeichnen Sie den Puls ein. Wird er erkannt? Was müsste das Gerät oder das Programm ändern? (2 P)



Ab jetzt tastet die Station den Taster jede Millisekunde ab (Zeitbasis aus Aufgabe 4). Ein Druck prellt: Der Kontakt schließt und öffnet einige Male, bevor er stabil bleibt. Ein Filter übernimmt eine Änderung erst nach drei gleichen Rohproben in Folge; der stabile Zustand beginnt mit 0. Serie ist die Zahl gleicher Rohproben in Folge bis einschließlich der aktuellen Probe.

3. Ergänzen Sie die Tabelle. Die ersten beiden Spalten sind ein Beispiel. (3 P)

Probe	1	2	3	4	5	6	7	8	9	10	11	12
Roh	0	1	0	1	1	1	1	0	1	0	0	0
Serie	1	1										
Stabil	0	0										

4. Die Druck-Freigabe-Erkennung aus der Vorlesung zählt jeden Übergang von „Warten auf Druck“ nach „Warten auf Freigabe“. Wie viele Drücke zählt sie auf den Rohproben, wie viele auf dem stabilen Zustand? Wie viele Millisekunden nach dem ersten Kontakt (Probe 2) meldet der Filter den Druck? Tipp: Zählen Sie die Wechsel von 0 auf 1. (2 P)

Aufgabe 4: Ein Timer-Interrupt als Zeitbasis (12 Punkte)

Für die Proben aus Aufgabe 3 löst ein Timer jede Millisekunde einen Interrupt aus. Es gilt das Papiermodell der Vorlesung: RV32 mit Machine Mode M und User Mode U, alle Traps gehen nach M, keine Delegation, keine verschachtelten Traps. Ursachencodes für diese Aufgabe: Interrupt 7 = Machine Timer, Interrupt 11 = Machine External, Exception 2 = Illegal Instruction, Exception 11 = `ecall` aus M.

1. Decodieren Sie mcause und bestimmen Sie das Handlerziel für den unterstützten Wert mtvec = `0x201`. Die erste Zeile ist ein Beispiel. Tipp: Die Bits 1:0 = 01 wählen Vectored mit BASE = `0x200`. Interrupts springen dann nach BASE plus viermal Code, Exceptions nach BASE. Mit mtvec = `0x200` (Direct) wäre das Ziel immer `0x200`. (3 P)

mcause	Bit 31	Code	Ursache	Ziel
`0x8000000B`	1	11	Machine External Interrupt	`0x22C`
`0x80000007`				
`0x0000000B`				
`0x00000002`				

2. Das Programm läuft in M und startet mit mstatus = `0x00000000`. Der Timer meldet bereits eine anstehende Anforderung (P = 1) und ist einzeln freigegeben (E = 1). Dann führt das Programm `li t1, 8` und `csrrs x0, mstatus, t1` aus. Welchen Wert hat mstatus danach, welches Feld ist gesetzt? Wird der Timer-Interrupt vor dem `csrrs` angenommen, und danach? (2 P)
3. Ergänzen Sie die Zustände beim Eintritt in den Handler und nach `mret`. Tipp: MIE hat die Maske `0x8`, MPIE die Maske `0x80`; das MPP-Feld für M (3) ergibt `0x1800`. (4 P)

Größe	vorher	im Handler	nach `mret`
aktueller Modus	M		
MIE (Bit 3)	1		
MPIE (Bit 7)	0		
MPP (Bits 12:11)	U		
mstatus	`0x00000008`		

4. Der Interrupt wird vor dem Befehl an `0x00400014` angenommen, mepc = `0x00400014`. Wohin kehrt `mret` zurück, wenn der Handler mepc nicht ändert? Stünde an `0x00400014` ein `ecall`, den der Handler vollständig bearbeitet: Welchen Wert schreibt er vor `mret` in mepc? Warum wäre derselbe Schritt beim Interrupt falsch? (3 P)

Aufgabe 5: Der Handler sichert seine Register (8 Punkte)

Der Timer-Handler zählt die Millisekunden im Speicherwort ticks an `0x1000` (Annahme). Er ruft keine Unterfunktion auf; die Quittierung des Timers ist hier weggelassen.

```
handler:
    addi sp, sp, -16
    sw   t0, 12(sp)
    sw   t1, 8(sp)
    li   t0, 0x1000          # Adresse von ticks
    lw   t1, 0(t0)
    addi t1, t1, 1
    sw   t1, 0(t0)           # ticks = ticks + 1
    lw   t1, 8(sp)
    lw   t0, 12(sp)
    addi sp, sp, 16
    mret
```

Der Interrupt trifft die Schleife aus Aufgabe 2 direkt nach `andi t1, t0, 3`; mepc zeigt auf `beq t1, zero, leer`. Der Tank ist halb voll, der Taster gedrückt: t0 = 5 (`0101`), t1 = 1. Außerdem gilt sp = `0x3000` und ticks = 41.

1. Welchen Wert hat sp im Handler? Ergänzen Sie den Stackframe nach den beiden `sw`. Die erste Spalte ist ein Beispiel. (2 P)

Adresse	`0x2FFC`	`0x2FF8`	`0x2FF4`	`0x2FF0`
Inhalt	t0 = 5			

2. Ergänzen Sie die Werte. Die erste Zeile ist ein Beispiel. (2 P)

Zeitpunkt	sp	t0	t1	ticks
vor dem Interrupt	`0x3000`	5	1	41
nach `sw t1, 0(t0)`				
nach `addi sp, sp, 16`				

3. In einer ersten Fassung fehlten `sw t1, 8(sp)` und `lw t1, 8(sp)`. Symptom: Bei halbvollem Tank blitzt Pixel 1 gelegentlich blau auf und ist danach wieder gelb. Welchen Wert hat t1 nach `mret` in dieser Fassung? Welchen Zweig nimmt die Schleife? Warum stimmt die Anzeige im nächsten Durchlauf wieder, und warum tritt der Fehler nur gelegentlich auf? (3 P)
4. t1 ist nach der Funktions-ABI ein temporäres Register. Warum muss der Handler es trotzdem sichern? (1 P)

Aufgabe 6: Den Wasserstand in Venus ausgeben (8 Punkte)

Ein Drucksensor liefert den Rohwert n. Der Wasserstand über der Ansaugöffnung der Pumpe ist $h = 3n - 4$ in cm; ein negativer Wert bedeutet, dass das Wasser unter der Öffnung steht. Das Programm gibt h über Venus-Dienste aus: Kennung in a0, Argument in a1; 1 = Print Integer (Zahl), 4 = Print String (Adresse eines nullterminierten Texts), 10 = Exit.

```
.data
n:       .word 17
praefix: .asciiz "Wasserstand in cm: "
.text
.globl main
main:
    la   a1, praefix
    li   a0, 4
    ecall                    # 1. ecall
    la   t0, n
    lw   t1, 0(t0)           # n laden
    slli t2, t1, 1           # 2n
    add  t2, t2, t1          # 3n
    addi a1, t2, -4          # h = 3n - 4
    li   a0, 1
    ecall                    # 2. ecall
    li   a0, 10
    ecall                    # 3. ecall
```

1. Ergänzen Sie die Belegung vor jedem `ecall` für n = 17. Die erste Zeile ist ein Beispiel. (2 P)

ecall	a0	a1	Dienst	Ausgabe
1.	4	Adresse von `praefix`	Print String	`Wasserstand in cm:`
2.				
3.				

2. Ergänzen Sie die Tabelle für weitere Rohwerte. Die erste Zeile ist ein Beispiel. (2 P)

n	t2 nach `slli`	t2 nach `add`	a1 = h	Konsolenausgabe
17	34	51	47	`Wasserstand in cm: 47`
2				
−3				

3. Bei einem Team erscheint für n = 17 und für n = 2 dieselbe große Zahl; der Text davor stimmt. Welche Größe hat das Programm offenbar statt n verrechnet? Welche Befehlsverwechslung führt typischerweise dazu, und wie lautet die Korrektur? (3 P)
4. Das Programm lädt n erst nach dem ersten `ecall`. Warum ist das robuster, als n vorher in t1 zu laden und über den Textdienst hinweg zu behalten? (1 P)

Aufgabe 7: Die Programme im Simulator prüfen (optional)

Teil A: Füllstandsanzeige in Ripes. Das Programm entspricht Aufgabe 2, verwendet aber die Basisadressen, die Ripes für die angelegten Geräte exportiert.

```
.text
.globl main
main:   li   s1, SWITCHES_0_BASE     # Schalterwort
        li   s2, LED_MATRIX_0_BASE   # Pixelbasis
loop:   lw   t0, 0(s1)
        andi t1, t0, 3
        beq  t1, zero, leer
        li   t3, 1
        beq  t1, t3, halb
        li   t3, 3
        beq  t1, t3, voll
        li   t2, 0x000000FF          # Blau
        j    zeigen
leer:   li   t2, 0x00FF0000          # Rot
        j    zeigen
halb:   li   t2, 0x00FFFF00          # Gelb
        j    zeigen
voll:   li   t2, 0x0000FF00          # Grün
zeigen: sw   t2, 4(s2)               # Pixel 1
        j    loop
```

1. Öffnen Sie Ripes ab Version 2.2, installiert oder unter ripes.dk, und wählen Sie über *Select Processor* den *RISC-V Single Cycle Processor*. Legen Sie im Tab *I/O* per Doppelklick ein Gerät *Switches* (mindestens drei Schalter) und ein Gerät *LED Matrix* an; stellen Sie dort *Height* = 1 und *Width* = 4 ein. Protokollieren Sie Version, Modell und die exportierten Werte von `SWITCHES_0_BASE` und `LED_MATRIX_0_BASE`. Heißen die Symbole anders, passen Sie die beiden `li`-Zeilen an.
2. Geben Sie das Programm im Tab *Editor* ein und starten Sie es im Tab *I/O* mit *Run*. In aktuellen Versionen setzt Schalter n das Bit n. Stellen Sie die Schalter 2 bis 0 auf `000`, `001`, `011`, `010` und `111`; sagen Sie die Farbe von Pixel 1 jeweils vorher.
3. Stellen Sie den Fehler aus Aufgabe 2.4 nach (Maske 7) und prüfen Sie danach Ihre Erweiterung aus Aufgabe 2.5.

Teil B: Wasserstand in Venus.

4. Öffnen Sie Venus unter thaumicmekanism.github.io/venus und geben Sie das Programm aus Aufgabe 6 im Tab *Editor* ein. Ein Klick auf den Tab *Simulator* übersetzt es; *Run* führt es aus, die Ausgabe erscheint unter *Console Output*. Gehen Sie nach *Reset* mit *Step* bis vor jeden `ecall` und vergleichen Sie a0 und a1 in der Registeransicht mit Ihrer Tabelle aus Aufgabe 6.1.
5. Ändern Sie n auf 2, auf –3 (`.word -3`) und auf 0; sagen Sie jede Zeile vorher. Stellen Sie danach den Fehler aus Aufgabe 6.3 nach (`la t1, n` statt `la t0, n` und `lw t1, 0(t0)`) und vergleichen Sie die Ausgabe mit $3 \cdot$ Adresse $- 4$.
6. In Ripes stehen laut Ripes-Dokumentation die Kennung in a7 und das erste Argument in a0; die Dienste zeigt das Menü *Help* unter *System calls*. Welche Zeilen müssten Sie für Ripes ändern?